# Especificação SDD: TaskAnalyzer

**Bootcamp III**, autor: Mateus Salvato
Fase 1: Especificação Técnica e Governança de Contexto (SDD & AI Harness)
Versão: **1.2** (transposta do Google Docs da Fase 1, conciliada com o enunciado da Fase 2 e ampliada com a infraestrutura da Fase 3; ver seção 6)

> *A especificação é o contrato. A IA é a executora. O desenvolvedor é o arquiteto e o homologador.*

## 1. Visão Geral e Contrato de Negócio

### 1.1 Propósito

O TaskAnalyzer é um módulo de análise de tarefas e produtividade. Ele recebe uma coleção de tarefas de um usuário ou equipe e produz métricas objetivas de desempenho de execução. O escopo é estritamente analítico: o módulo não cria, edita nem exclui tarefas.

Ele permite identificar de forma automatizada e confiável:

- o tempo médio de conclusão das tarefas, geral e por prioridade;
- a taxa de atraso das tarefas concluídas, geral e por prioridade;
- a quantidade de tarefas analisadas, geral e por prioridade;
- indicadores consolidados por prioridade (`baixa`, `media`, `alta`).

É um componente puro: recebe uma lista em memória e devolve uma estrutura de métricas, sem efeitos colaterais, sem persistência e sem dependências externas.

### 1.2 Interface pública

```
def analyze_tasks(tarefas: list[dict[str, Any]]) -> TaskAnalysis: ...

class TaskValidationError(ValueError): ...
```

Módulo: `src/task_analyzer.py`.

### 1.3 Entradas

Lista de dicionários; cada tarefa possui:

| Campo | Tipo | Obrigatório | Restrições |
|---|---|---|---|
| `id_tarefa` | `int` | Sim | Inteiro positivo (> 0), único na lista |
| `titulo` | `str` | Sim | Não pode ser vazio ou só espaços |
| `data_criacao` | `datetime` ou `str` ISO 8601 | Sim | UTC |
| `data_inicio` | `datetime` ou `str` ISO 8601 | Não | Se ausente, considera-se igual a `data_criacao` |
| `data_conclusao` | `datetime` ou `str` ISO 8601 | Condicional | Obrigatória quando `status = "concluida"` |
| `prazo` | `datetime` ou `str` ISO 8601 | Sim | Data/hora limite acordada |
| `prioridade` | `str` (enum) | Sim | `"baixa"`, `"media"` ou `"alta"` |

| | | | |
|---|---|---|---|
| `status` | `str` (enum) | Sim | `"concluida"`, `"pendente"` ou `"cancelada"` |

### 1.4 Saídas

Dicionário `TaskAnalysis`:

| Chave | Tipo | Descrição |
|---|---|---|
| `total_tarefas` | `int` | Quantidade de tarefas analisadas |
| `total_concluidas` | `int` | Tarefas com `status = "concluida"` |
| `total_pendentes` | `int` | Tarefas com `status = "pendente"` |
| `total_canceladas` | `int` | Tarefas com `status = "cancelada"` |
| `tempo_medio_conclusao_horas` | `float` | Média de `data_conclusao − data_inicio`, em horas, das concluídas |
| `taxa_atraso_percentual` | `float` | % de concluídas com `data_conclusao > prazo` |
| `sem_dados` | `bool` | `True` quando não há tarefa concluída para calcular médias |
| `indicadores_por_prioridade` | `dict` | Chaves `baixa`, `media`, `alta` (sempre presentes), cada uma com `total_tarefas`, `total_concluidas`, `tempo_medio_conclusao_horas` e `taxa_atraso_percentual` |

### 1.5 Regras de negócio

- **RN01:** só tarefas `concluida` entram no tempo médio e na taxa de atraso.
- **RN02:** tempo de conclusão = `data_conclusao − data_inicio` (ou `data_criacao`, se `data_inicio` ausente).
- **RN03:** uma tarefa está em atraso quando `data_conclusao > prazo`. Pendentes e canceladas não entram nesse cálculo.
- **RN04:** datas inconsistentes ou ausentes quando obrigatórias disparam `TaskValidationError` com mensagem clara. Nunca há falha silenciosa.
- **RN05:** divisão por zero (nenhuma concluída no conjunto ou numa prioridade) é prevenida: o valor retornado é `0.0` e `sem_dados` indica a ausência de dados.
- **RN06:** todas as datas são interpretadas e comparadas em UTC. Datas sem fuso são tratadas como UTC.
- **RN07:** tempos médios e percentuais têm 2 casas decimais.

### 1.6 Validações (`TaskValidationError`)

1. Entrada que não é lista, ou item que não é dicionário.
2. Campo obrigatório ausente ou `None`.
3. `id_tarefa` que não é inteiro positivo, ou repetido.
4. `titulo` vazio ou só espaços.
5. `prioridade` ou `status` fora do enum.
6. Data em formato inválido.
7. Tarefa `concluida` sem `data_conclusao`.
8. `data_conclusao` anterior a `data_criacao`.
9. `data_conclusao` anterior a `data_inicio`.

## 2. Cenários de Aceite (Dado–Quando–Então)

### Cenário 1: Sucesso (métricas corretas)

- **Dado** um conjunto de tarefas válidas com prioridades, prazos e status consistentes
- **Quando** o TaskAnalyzer for executado
- **Então** retorna `total_tarefas`, `total_concluidas`, `total_pendentes`, `tempo_medio_conclusao_horas` e `taxa_atraso_percentual` corretos, de forma geral e por prioridade.

### Cenário 2: Exceção / Erro (entradas inválidas)

- **Dado** uma entrada com datas inválidas (ex.: `data_conclusao < data_criacao` ou `data_conclusao < data_inicio`)
- **Quando** o TaskAnalyzer for executado
- **Então** dispara `TaskValidationError` com mensagem clara identificando a tarefa.

### Cenários adicionais (casos de borda)

- Lista vazia ⇒ métricas zeradas, `sem_dados = True`, sem exceção.
- Apenas tarefas pendentes ⇒ médias `0.0`, sem exceção.
- Tarefa pendente ou cancelada ⇒ conta no total, mas não no tempo médio nem na taxa de atraso.
- Prioridade fora do enum ⇒ `TaskValidationError`.

## 3. Planejamento do Test Harness

Cenários de aceite → casos de teste → testes `pytest` em `tests/test_harness.py` (com fixtures) → execução a cada alteração → evidência (`pass/fail`) para a homologação humana. Há no mínimo um teste automatizado por cenário.

## 4. Estrutura do repositório

```
sdd-task-analyzer/
├── .github/
│   └── workflows/
│       └── ci.yml                    # Motor CI/CD (GitHub Actions)
├── Dockerfile                        # Empacotamento (python:3.11-slim)
├── docker-compose.yml                # Orquestração local dos testes
├── .dockerignore
├── CONTEXT_RULES.md                  # Base de governança (Fase 1)
├── specs/
│   ├── system_design_document.pdf    # Esta especificação em PDF
│   ├── requirements.yaml             # Requisitos executáveis e rastreabilidade
│   └── task_analyzer_spec.md         # Fonte editável desta especificação
├── tests/
│   └── test_harness.py               # Suíte de testes (Fase 2)
├── src/
│   └── task_analyzer.py              # Código gerado por IA e homologado (Fase 2)
├── .gitignore
├── requirements.txt
└── README.md
```

## 5. Infraestrutura e Entrega Contínua (Fase 3)

### 5.1 Pipeline de CI (`.github/workflows/ci.yml`)

- **Gatilhos:** `push` e `pull_request` nas branches `main` e `develop`.
- **Ambiente:** `ubuntu-latest` com Python 3.11.
- **Preparação:** `pip install -r requirements.txt`.
- **Execução:** `pytest tests/test_harness.py -v`.
- **Job extra:** build da imagem Docker e execução da suíte dentro do container.
- Um PR só pode ser mesclado na `main` com o pipeline verde (*green build*).

### 5.2 Containerização

- `Dockerfile`: imagem base `python:3.11-slim`, instala as dependências, copia o código e define `pytest tests/test_harness.py -v` como comando de entrada (`CMD`). Roda com usuário sem privilégios.
- `docker-compose.yml`: serviço `tests`; toda a suíte roda com um único comando: `docker compose run --rm tests`.

### 5.3 Critério de aceite da Fase 3

- **Dado** um commit na `main` ou um PR aberto
- **Quando** o pipeline de CI executar
- **Então** os jobs `test` e `docker` terminam com 100% dos testes aprovados.

## 6. Histórico de versões

| Versão | Mudança | Motivo |
|---|---|---|
| 1.0 | Especificação original (Google Docs, Fase 1) | — |
| 1.1 | Nomes da interface: `analyze_tasks` e `TaskValidationError` (antes "ex.: TarefaInvalidaError") | Exigência do enunciado da Fase 2 |
| 1.1 | Divisão por zero passa a retornar `0.0` + `sem_dados`, em vez de `DivisaoPorZeroError` | A v1.0 era contraditória (Cenário 2 pedia exceção; cenário adicional pedia métricas zeradas para lista vazia). O enunciado da Fase 2 exige `0.0` sem exceção. Conflito sinalizado pela IA e decidido na homologação humana |
| 1.1 | Saída ganha `total_concluidas`, `total_pendentes`, `total_canceladas` e sufixo `_horas` em `tempo_medio_conclusao`; `quantidade_tarefas` passa a se chamar `total_tarefas` | Nomes exigidos pelo enunciado da Fase 2 |
| 1.1 | Datas inconsistentes disparam exceção (fail-fast) em vez de "ignorar e reportar" | Cenário 2 da própria v1.0 exige disparar exceção |
| 1.1 | Datas aceitas também como `str` ISO 8601 | A v1.0 já citava ISO 8601 |
| 1.2 | Estrutura do repositório ampliada (`.github/`, `Dockerfile`, `docker-compose.yml`, `specs/system_design_document.pdf`, `specs/requirements.yaml`) e nova seção 5 | Exigência do enunciado da Fase 3. A mudança de estrutura foi aprovada pelo arquiteto humano, como exige o CONTEXT_RULES §2.3 |